



香港中文大學
The Chinese University of Hong Kong

CENG3420

Lab 2-1: RISC-V RV32I Assembler

Libo Shen, Xiaoman Yang
Department of Computer Science & Engineering
Chinese University of Hong Kong
{lbshen24, xmyang25}@cse.cuhk.edu.hk

Spring 2026

- ① Introduction
- ② Introduction to RV32I
- ③ RISC-V-LC Code Specifications
- ④ RISC-V-LC Assembler
- ⑤ Lab 2-1 Assignment

Introduction

Use C programming language to finish lab assignments in following weeks.

- Lab 2.1 – implement an RISC-V-LC Assembler
- Lab 2.2 – implement an RISC-V-LC ISA Simulator
- Lab 3.x – implement an RISC-V-LC Simulator

NOTICE

Lab2 & Lab3 are challenging!

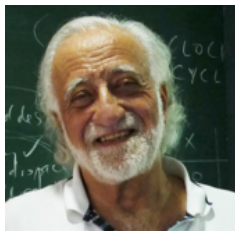
Once you have passed Lab2 & Lab3, you will be more familiar with RV32I & a basic implementation!

- Assembly language – symbolic (we have learned in Lab1)
- Machine language – binary
- **Assembler** is a program that
 - turns symbols into machine instructions
 - e.g. riscv64-unknown-elf-as
- **Simulator** is a program that
 - mimics the behavior of a processor
 - usually written in high-level language
 - e.g. spike

Introduction

Our Lab2 & Lab3 are Inspired by LC-3b

- LC-3b: **Little Computer 3, b** version.
- Relatively simple instruction set
- Used in CS & CE teaching courses
- Developed by Yale Patt@UT & Sanjay J. Patel@UIUC



In Lab2-2, our RV32I ISA Simulator integrates

- RISC-V 32 general-purpose registers
- 32-bit data and address
- 25+ instructions (including pseudo instructions)

In Lab3, 4 more special-purpose registers will be added:

- Program Counter ([PC](#))
- Instruction Register ([IR](#))
- Memory Access Register ([MAR](#))
- Memory Data Register ([MDR](#))

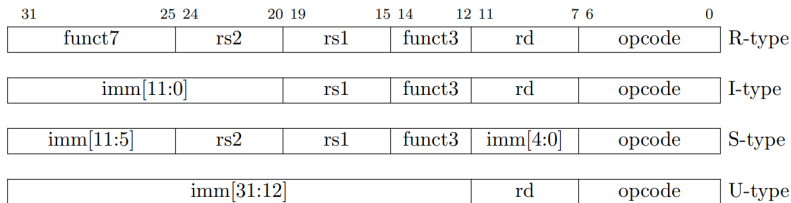
Introduction to RV32I

- The instruction encoding width is 32-bit.
- Each instruction is aligned with a **four-byte** boundary in memory.
- RV32I only manipulates integer numbers and no multiplication or division.
- Our labs are based on the official RISC-V ISA manual:
<https://www.cse.cuhk.edu.hk/~byu/CENG3420/2025Spring/doc/RV32-reference-1.pdf>
<https://github.com/riscv/riscv-isa-manual/releases/download/Ratified-IMAFDQC/riscv-spec-20191213.pdf>

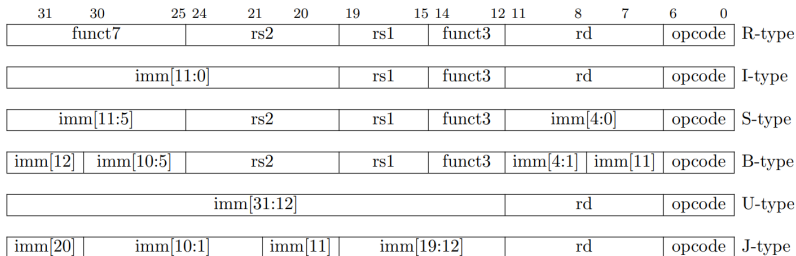
- **Integer Computational Instructions**
- **Control Transfer Instructions**
- **Load and Store Instructions**
- Memory Ordering Instructions
- Environment Call and Breakpoints
- HINT Instructions

Introduction to RV32I

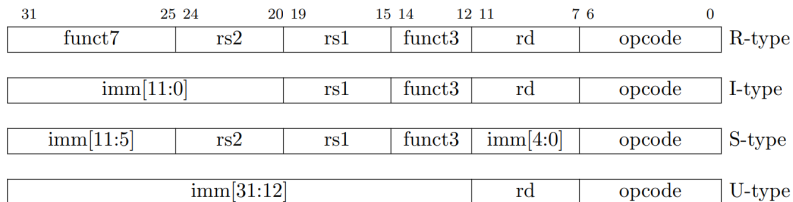
- Four core instruction formats:
Register Type, Immediate Type, Store Type, Upper Immediate Type



- Two variants of the instruction formats



Introduction to RV32I



The fields of an instruction

- opcode: indicates the category of the Instruction
 - 0010011 integer register-immediate, 0110011 integer register-register, 1100011 conditional branches,
- funct7, funct3: distinguishes difference instructions in one category
- rs1, rs2: source registers
- rd: destination register
- imm: immediate value
 - imm[11:5]: the bits {11, 10, 9, 8, 7, 6, 5} of the immediate value

- Integer Register-Immediate Instructions
- addi (Add Immediate), slti (Set Less Than Immediate), andi (AND Immediate), ori (OR Immediate), xori (XOR Immediate), slli (Shift Left Logical Immediate), srli (Shift Right Logical Immediate), srai (Shift Right Arithmetic Immediate), lui (Load Upper Immediate)

Integer Register-Register Instructions

- add (Add), slt (Set Less Than), and (AND), or (OR), xor (XOR), sll (Shift Left Logical), srl (Shift Right Logical), sub (Subtract), sra (Shift Right Arithmetic)

Introduction to RV32I

Integer Register-Immediate Instructions

31	20	19	15	14	12	11	7	6	0	
imm[11:0]			rs1		funct3		rd		opcode	
12			5		3		5		7	
I-immediate[11:0]			src		ADDI/SLTI[U]		dest		OP-IMM	
I-immediate[11:0]			src		ANDI/ORI/XORI		dest		OP-IMM	

31	25 24	20 19	15 14	12 11	7 6	0
imm[11:5]		imm[4:0]	rs1	funct3	rd	opcode
7		5	5	3	5	7
0000000		shamt[4:0]	src	SLLI	dest	OP-IMM
0000000		shamt[4:0]	src	SRLI	dest	OP-IMM
0100000		shamt[4:0]	src	SRAI	dest	OP-IMM

31	12	11	7	6	0
imm[31:12]			rd	opcode	
20			5	7	
U-immediate[31:12]			dest	LUI	
U-immediate[31:12]			dest	AUIPC	

Introduction to RV32I

Integer Register-Register Instructions

31	25 24	20 19	15 14	12 11	7 6	0
funct7	rs2	rs1	funct3	rd	opcode	
7	5	5	3	5	7	
0000000	src2	src1	ADD/SLT/SLTU	dest	OP	
0000000	src2	src1	AND/OR/XOR	dest	OP	
0000000	src2	src1	SLL/SRL	dest	OP	
0100000	src2	src1	SUB/SRA	dest	OP	

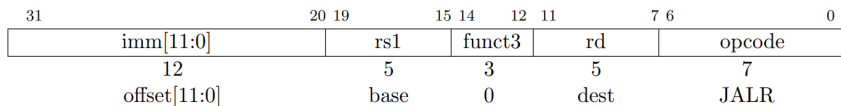
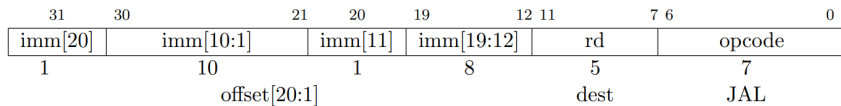
- Unconditional Jumps
- jal (Jump and Link), jalr (Jump and Link Register)

Conditional Branches

- beq (Branch if Equal), bne (Branch if Not Equal), blt (Branch if Less Than), bge (Branch if Greater or Equal)

Introduction to RV32I

Unconditional Jumps



Introduction to RV32I

Conditional Branches

31	30	25	24	20	19	15	14	12	11	8	7	6	0
imm[12]	imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	opcode						
1	6	5	5	3	4	1	7						
offset[12 10:5]		src2	src1	BEQ/BNE	offset[11 4:1]		BRANCH						
offset[12 10:5]		src2	src1	BLT[U]	offset[11 4:1]		BRANCH						
offset[12 10:5]		src2	src1	BGE[U]	offset[11 4:1]		BRANCH						

Load

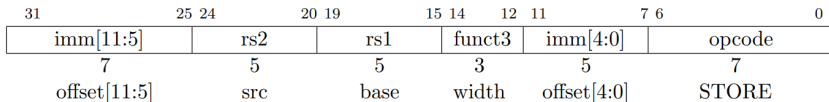
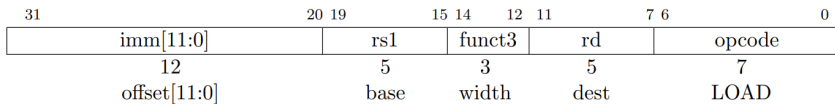
- lb (Load Byte), lh (Load Halfword), lw (Load Word)

Store

- sb (Store Byte), sh (Store Halfword), sw (Store Word)

Introduction to RV32I

Load and Store Instructions



RISCV-LC Code Specifications

NOTICE

To make labs easy, some self-defined directives are added.

- Label specification: no colon and one code line cannot contain only labels.
- No .data and .text directives
- Add one pseudo instruction: LA
- Add one self-customized directive: .FILL
- Add one halt instruction: HALT

Label specification: no colon and one code line cannot contain only labels.

```
1      la a0, AL
2      lw a0, 0(a0)
3      blt a0, zero, L1
4 ▼ L1  addi a7, a0, 13
5      bge zero, a7, L1
```

Specification examples

Add one pseudo instruction: **la**.

Loads a 32 bit address from memory.

la is translated to two RV32I instructions: **lui** and **addi**.

```
1      la a0, A # lui a0, 0x0; addr = 0x0
2      # addi a0, a0, 0x8; addr = 0x4
3      A .FILL -2 # addr = 0x8
```

Translate **la** to **lui** and **addi**

.FILL is similar to .byte, .word, etc.

```
30
31    AL    .FILL -2
32    BL    .FILL -9
33
```

.FILL directive

- .byte: Allocates a single byte of data in memory.
- .word: Allocates a single word of data (usually 4 bytes) in memory.
- .fill: Allocates a 32 bit data in memory.

RISCV-LC Assembler

Lab 2-1 Assignment

Pre-requisites

- Learn Linux/MacOS terminal commands, and we prefer using Linux/MacOS
- If you are using windows system, we suggest you to use wsl: <https://documentation.ubuntu.com/wsl/stable/howto/install-ubuntu-wsl2/>
- Install git in your computer: `sudo apt-get install git`
- Learn git from the reference: <https://git-scm.com/docs/gittutorial>
- Learn Makefile from the reference: <https://makefiletutorial.com/>

Lab 2-1 Assignment

Pre-requisites – Get RISC-V LC Assembler

Please visit the website for more information:

<https://github.com/seanzw/CUHK-CENG3420-Lab>

Get the RV32I Assembler

In the terminal of your computer, type these commands:

- `git clone https://github.com/seanzw/CUHK-CENG3420-Lab.git`
- `cd CUHK-CENG3420-Lab`
- `git checkout lab2.1`

Compile (Linux/MacOS environment is suggested)

- `make`

Run the assembler

- `./asm benchmarks/isa.asm isa.bin` # you can check the output machine code: isa.bin if you have implemented the assembler

Now, let's take a live demo by walking through the code repo to learn the assembler.

- Know the codes organizations.
- `addi` example in **asm.c**
- `add` example in **asm.c**
- `beq` example in **asm.c**
- `lb` example in **asm.c**

Lab 2-1 Assignment

Finish the RV32I assembler including 25 instructions in asm.c as follows

- Integer Register-Immediate Instructions: slli, xori, srli, srai, ori, andi, lui
- Integer Register-Register Operations: sub, sll, xor, srl, sra, or, and
- Unconditional Jumps: jalr, jal
- Conditional Branches: bne, blt, bge
- Load and Store Instructions: lb, lh, lw, sb, sh, sw

These unimplemented codes are commented with [Lab2-1 assignment](#)

Verification of Implementations

Verify your codes with these benchmarks (inside the `benchmarks` directory)

- `isa.asm`
- `count10.asm`
- `swap.asm`

Compare your output with `.bin` file. If both of them are the same, you are correct!

A quick verification command in the terminal: `make validate # generate reports inside the tools directory.`

Submission Method:

Submit a **zip file** including **source codes** and a **report** **after** the **whole lectures of Lab2** into **Blackboard**.

Tips

Inside `docs`, there are three valuable documents for your reference!

- `opcodes-rv32i`: RV32I opcodes
- `riscv-spec-20191213.pdf`: RV32I specifications
- `risc-v-asm-manual.pdf`: RV32I assembly programming manual